Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	27-01-2026		

Experiment 2: Binary Classification using Naïve Bayes and K-Nearest Neighbors

1 Aim and Objective

To implement Naïve Bayes and K-Nearest Neighbors (KNN) classifiers for a binary classification problem using the Spambase dataset, evaluate them using multiple performance metrics, and analyze their behavior.

2 Dataset Description

The experiment uses the **Spambase Dataset** containing 57 features related to word and character frequencies in emails. The target variable is `class`, where 1 denotes spam and 0 denotes ham.

3 Preprocessing Steps

Based on the implementation, the following preprocessing steps were carried out:

1. **Data Cleaning**: Duplicate rows were identified using `df.duplicated()` and removed with `df.drop_duplicates()`.
2. **Missing Values**: Handled by checking for null entries using `df.isnull().sum()`.
3. **Feature Scaling**: Since KNN is distance-dependent, `StandardScaler` was used to normalize the features.
4. **Data Splitting**: The dataset was split into training and testing sets using `train_test_split`.

4 Implementation Details

The following code snippets illustrate the core implementation of the classifiers:

Important Libraries to be included

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
import math
from matplotlib import rcParams
from sklearn.model_selection import train_test_split, StratifiedKFold,
    GridSearchCV, RandomizedSearchCV,cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, roc_curve, auc
from sklearn.model_selection import learning_curve
import time
from sklearn.metrics import  classification_report, roc_auc_score,
    average_precision_score
from scipy.stats import f_oneway

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

1. Load the dataset

```
# loading dataset
df = pd.read_csv("spambase_csv.csv")

# Seeing first 5 rows.
df.head()

# Checking the data types of each features of the dataset
df.info()
```

2. Perform data preprocessing (handling missing values and scaling)

```
# Checking if any duplicate rows are found.
print("Duplicates in data: ", any(df.duplicated()))

# Remove duplicates rows.
df = df.drop_duplicates()

# Checking the missing and null values...for train data
any(df.isnull().sum())
```

```
# Separate features (X) and target (y)
X = df.drop('class', axis=1)
y = df['class']
```

3. Perform Exploratory Data Analysis (EDA)

```
# Statistical summary for train data
df.describe()

print("Shape of dataset: ",df.shape)

# Boxplot after standard scalar for outliers.

cols_per_plot = 10  # number of features in each subplot
total_cols = X.shape[1]
num_plots = math.ceil(total_cols / cols_per_plot)

fig, axes = plt.subplots(nrows=num_plots, ncols=1, figsize=(16, num_plots*4))

for i in range(num_plots):
    start_col = i * cols_per_plot
    end_col = min((i+1) * cols_per_plot, total_cols)
    subset = X.iloc[:, start_col:end_col]  # select subset of columns

    sns.boxplot(data=subset, ax=axes[i])
    axes[i].set_title(f"Boxplot for Features {start_col+1} to {end_col}")
    axes[i].set_xlabel("Features")
    axes[i].set_ylabel("Values")
    axes[i].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Handling outliers by using capping.
cols = df.drop(columns='class', axis = 1)
for col in cols:
    upper_limit = df[col].quantile(0.95)
    lower_limit = df[col].quantile(0.05)
    df[col] = np.where(df[col]>upper_limit, upper_limit, np.where(df[col]<
   lower_limit, lower_limit, df[col]))


# Separate features (X) and target (y) again after handling outliers.
X = df.drop('class', axis=1)
y = df['class']

# Correlation analysis
corr = df.corr()

# Heatmap plot for correlation analysis
plt.figure(figsize=(12, 10))
sns.heatmap(
    corr,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    square=True,
```

```
    annot_kws={"size": 8}
)
plt.title("Correlation Heatmap")
plt.show()
```

4. Visualize class distribution and feature behavior

```
# Class Distribution Plot
sns.countplot(data=df,x="class")

# Feature vs. target scatter plots
# Select numerical features
num_features = df.select_dtypes(include=['int64', 'float64']).columns

# Number of columns
cols = 3
rows = math.ceil(len(num_features) / cols)

# Create subplots
fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 4*rows))
axes = axes.flatten()

# Plot each feature
for i, col in enumerate(num_features):
    sns.scatterplot(x=df[col], y=df["class"], ax=axes[i])
    axes[i].set_title(f"{col} vs Target")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Class")

# Remove empty subplots
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.show()

# Histogram
cols = df.columns
ncols = 3
nrows = math.ceil(len(cols) / ncols)

fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(15, 4 * nrows))
axes = axes.flatten()

bxwidth = 1

for i, col in enumerate(cols):
    sns.histplot(df[col], bins=20, ax=axes[i])
    axes[i].set_title(f"{col.upper()} Distribution", pad=20)
    axes[i].set_xlabel(col, labelpad=0)
    axes[i].set_ylabel("Frequency", labelpad=10)
    for spine in axes[i].spines.values():
        spine.set_linewidth(bxwidth)

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
```

```
plt.subplots_adjust(hspace=1.5, wspace=1.3)
plt.tight_layout()
plt.show()
```

5. Split the dataset into training and testing sets

```
# Checking the class balance
print("Class 1 has : ", df[df['class'] == 1].shape)
print("Class 0 has : ", df[df['class'] == 0].shape)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2,
    random_state=42, stratify=y)
```

6. Train Naive Bayes variants

```
# Standardize numerical features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

gnb = GaussianNB()
mnb = MultinomialNB()
bnb = BernoulliNB()

train_time = [], pred_time = [], accuracy = [], precision = []
recall = [], f1 = [], specificity = [], false_positive_rate = [], report = []

def calculate_metric(models):
    train_time.clear()
    pred_time.clear()
    accuracy.clear()
    precision.clear()
    recall.clear()
    f1.clear()
    specificity.clear()
    false_positive_rate.clear()
    report.clear()
    for model in models:
        start = time.time()
        if model == mnb:
            model.fit(X_train, y_train)
        else:
            model.fit(X_train_scaled, y_train)
        train_time.append(time.time() - start)

        start = time.time()
        if model == mnb:
            y_pred = model.predict(X_test)
        else:
            y_pred = model.predict(X_test_scaled)
        pred_time.append(time.time() - start)

        accuracy.append(accuracy_score(y_test, y_pred))
        precision.append(precision_score(y_test, y_pred))
```

```
        recall.append(recall_score(y_test, y_pred))
        f1.append(f1_score(y_test, y_pred))

        cm = confusion_matrix(y_test, y_pred)
        tn, fp, fn, tp = cm.ravel()

        specificity.append(tn / (tn + fp))
        false_positive_rate.append(fp / (fp + tn))

        report.append(classification_report(y_test,y_pred))

models = [gnb, mnb, bnb]
methods = ["Gaussian NB", "Multinomial NB", "Bernoulli NB"]
calculate_metric(models)

NB_table = pd.DataFrame({
    "Metric":["Accuracy", "Precision", "Recall", "F1 Score", "Specificity", "
    Training Time (s)"],
    "Gaussian NB": [accuracy[0], precision[0], recall[0], f1[0], specificity[0],
    train_time[0]],
    "Multinomial NB": [accuracy[1], precision[1], recall[1], f1[1], specificity
    [1], train_time[1]],
    "Bernoulli NB": [accuracy[2], precision[2], recall[2], f1[2], specificity[2],
    train_time[2]],
})
NB_table

# Naive Bayes - Bar Graph
nb_scores = [
    precision_score(y_test, gnb.predict(X_test)),
    precision_score(y_test, mnb.predict(X_test)),
    precision_score(y_test, bnb.predict(X_test))
]

# Bar plot
plt.figure(figsize=(5,4))
plt.bar(['GaussianNB', 'MultinomialNB', 'BernoulliNB'], nb_scores)
plt.ylabel('Precision')
plt.title('Naive Bayes Model Comparison')
plt.grid(axis='y', alpha=0.3)
plt.show()
```

7. Train a baseline KNN classifier

```
knn = KNeighborsClassifier()
knn.fit(X_train_scaled,y_train)
y_pred_knn = knn.predict(X_test_scaled)

# Accuracy vs k plot
k_values = range(1, 21)
accuracies = []

for k in k_values:
  knn = KNeighborsClassifier(n_neighbors=k)
  knn.fit(X_train_scaled, y_train)
  accuracies.append(knn.score(X_test_scaled, y_test))
```

```
plt.plot(k_values, accuracies, marker='o')
plt.xlabel('k value')
plt.ylabel('Accuracy')
plt.title('Accuracy vs k')
plt.show()
```

8. Perform hyperparameter tuning for KNN using 5-Fold Cross-Validation

```
# Stratified K fold
skf=StratifiedKFold(n_splits=5,shuffle=True,random_state=42)
cv_scores_base=cross_val_score(
    knn,
    X_train_scaled,
    y_train,
    cv=skf,
    scoring='precision'
)
print("Base KNN CV Precision:",cv_scores_base.mean())

# Grid Search CV
param_grid = {
    'n_neighbors': list(range(1, 31, 2)),
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan']
}

grid = GridSearchCV(
    knn,
    param_grid,
    cv=skf,
    scoring='precision',
    n_jobs=-1
)

grid.fit(X_train_scaled, y_train)

best_k = grid.best_params_['n_neighbors']

print("Grid Best Params:", grid.best_params_)
print("Best k (n_neighbors):", best_k)
print("Grid Best CV Precision:", grid.best_score_)

# Randomized Search CV
from scipy.stats import randint
param_dist={
    'n_neighbors':randint(1,30),
    'weights':['uniform','distance'],
    'metric':['euclidean','manhattan']
}
rand = RandomizedSearchCV(
    knn,
    param_distributions=param_dist,
    n_iter=15,
    cv=skf,
    scoring='precision',
    random_state=42,
    n_jobs=-1
```

```
)
rand.fit(X_train_scaled,y_train)
best_k = rand.best_params_['n_neighbors']
print("Random Best Params:",rand.best_params_)
print("Best k (n_neighbors):", best_k)
print("Random Best CV Precision:",rand.best_score_)

# Final KNN
best_params = rand.best_params_

knn_final = KNeighborsClassifier(
    n_neighbors=best_params['n_neighbors'],
    weights=best_params['weights'],
    metric=best_params['metric']
)
optimal_k = best_params['n_neighbors']
print("Optimal K: ", optimal_k)
```

9. Train optimized KNN models using KDTree and BallTree

```
knn_kd = KNeighborsClassifier(
    n_neighbors = optimal_k,
    weights = best_params['weights'],
    metric = best_params['metric'],
    algorithm = 'kd_tree'
)

knn_bt = KNeighborsClassifier(
    n_neighbors = optimal_k,
    weights = best_params['weights'],
    metric = best_params['metric'],
    algorithm = 'ball_tree'
)
models = [knn_final, knn_kd, knn_bt]
calculate_metric(models)

KNN_table = pd.DataFrame({
    "Metric":["optimal k", "Accuracy", "Precision", "Recall", "F1 Score", "
    Specificity", "false_positive_rate", "Training Time", "prediction Time"],
    "Generic KNN": [optimal_k, accuracy[0], precision[0], recall[0], f1[0],
    specificity[0],false_positive_rate[0], train_time[0], pred_time[0]],
    "KD Tree": [optimal_k, accuracy[1], precision[1], recall[1], f1[1],
    specificity[1], false_positive_rate[1], train_time[1], pred_time[1]],
    "Ball Tree": [optimal_k, accuracy[2], precision[2], recall[2], f1[2],
    specificity[2], false_positive_rate[2], train_time[2], pred_time[2]]
})
KNN_table

# KNN - Bar Graph
nb_scores = [
    precision_score(y_test, knn_final.predict(X_test)),
    precision_score(y_test, knn_kd.predict(X_test)),
    precision_score(y_test, knn_bt.predict(X_test))
]

# Bar plot
plt.figure(figsize=(5,4))
```

```
plt.bar(['Generic KNN', 'KD Tree KNN', 'Ball Tree KNN'], nb_scores)
plt.ylabel('Precision')
plt.title('KNN Model Comparison')
plt.grid(axis='y', alpha=0.3)
plt.show()

# Training vs Validation Accuracy Plot
train_acc = []
val_acc = []
k_values=range(1,31,2)
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled,y_train)
    train_acc.append(knn.score(X_train_scaled,y_train))
    val_acc.append(knn.score(X_test_scaled,y_test))

plt.plot(k_values,train_acc,label='Training Accuracy')
plt.plot(k_values,val_acc,label='Validation Accuracy')
plt.xlabel('k value')
plt.ylabel('Accuracy')
plt.title('Training vs Validation Accuracy')
plt.legend()
plt.show()

plt.savefig('training_vs_validation.png', dpi=600, bbox_inches='tight')
```

10. Evaluate all models using multiple metrics

```
# Cross-Validation Scores
gnb_scores = cross_val_score(GaussianNB(), X_train, y_train, cv = skf, scoring='
    precision')
mnb_scores = cross_val_score(MultinomialNB(), X_train, y_train, cv = skf, scoring=
    'precision')
bnb_scores = cross_val_score(BernoulliNB(), X_train, y_train, cv = skf, scoring='
    precision')
knn_kd_scores = cross_val_score(knn_kd, X_train_scaled, y_train, cv = skf, scoring
    ='precision')
knn_bt_scores = cross_val_score(knn_bt, X_train_scaled, y_train, cv = skf, scoring
    ='precision')

# One-Way ANOVA Test
F_stat,p_value=f_oneway(
    gnb_scores,
    mnb_scores,
    bnb_scores,
    knn_kd_scores,
    knn_bt_scores
)

print("F-statistic:",F_stat)
print("p-value:",p_value)

print("Gaussian NB Mean precision:",gnb_scores.mean())
print("Multinomial NB Mean precision:",mnb_scores.mean())
print("Bernoulli NB Mean precision:",bnb_scores.mean())
print("KNN KDTree Mean precision:",knn_kd_scores.mean())
print("KNN BallTree Mean precision:",knn_bt_scores.mean())
```

```
best_model=max(
    [
        ("Gaussian NB",gnb_scores.mean()),
        ("Multinomial NB",mnb_scores.mean()),
        ("Bernoulli NB",bnb_scores.mean()),
        ("KNN KDTree",knn_kd_scores.mean()),
        ("KNN BallTree",knn_bt_scores.mean())
    ],
    key=lambda x:x[1]
)

print("Best Model:",best_model)
```

5 Visualizations

This section presents the visual analysis of the model's performance, including class distribution, feature behavior, and classification results across all models.

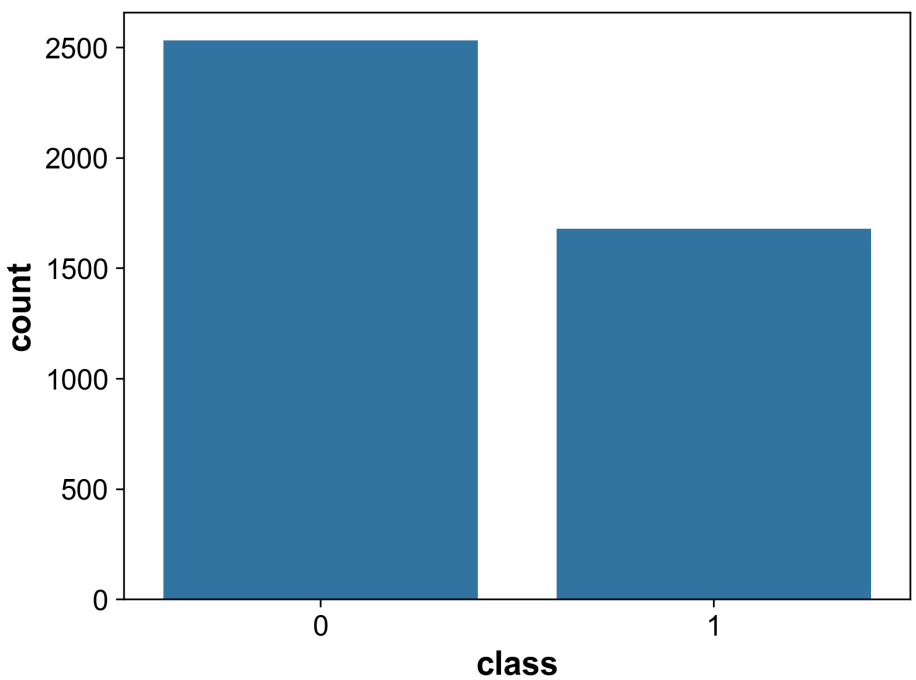

Figure 1: Class Distribution Plots (Spam vs Ham)

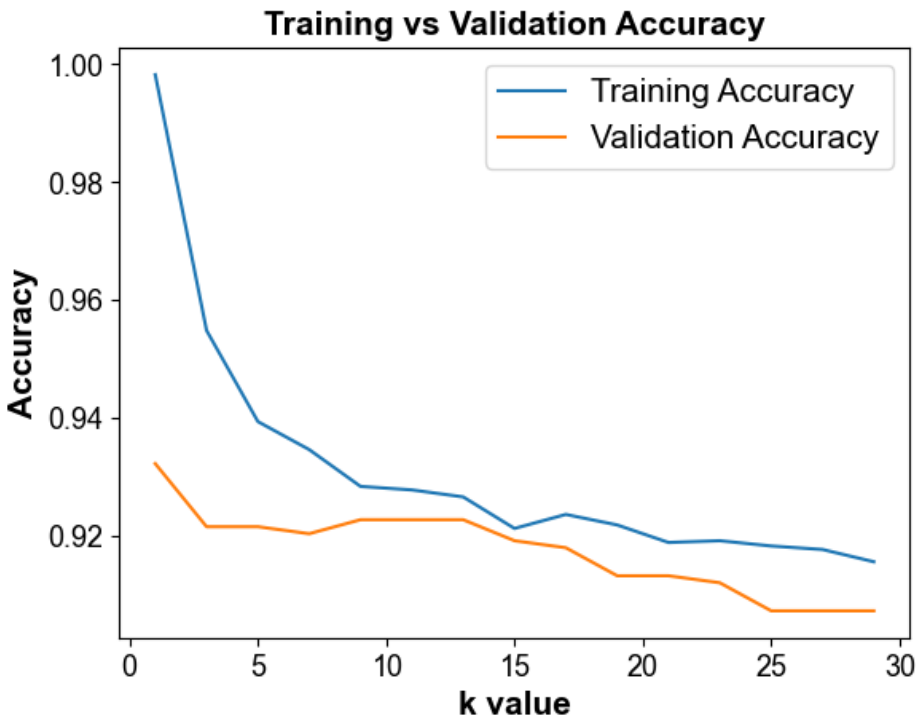

Figure 2: Training vs Validation Accuracy

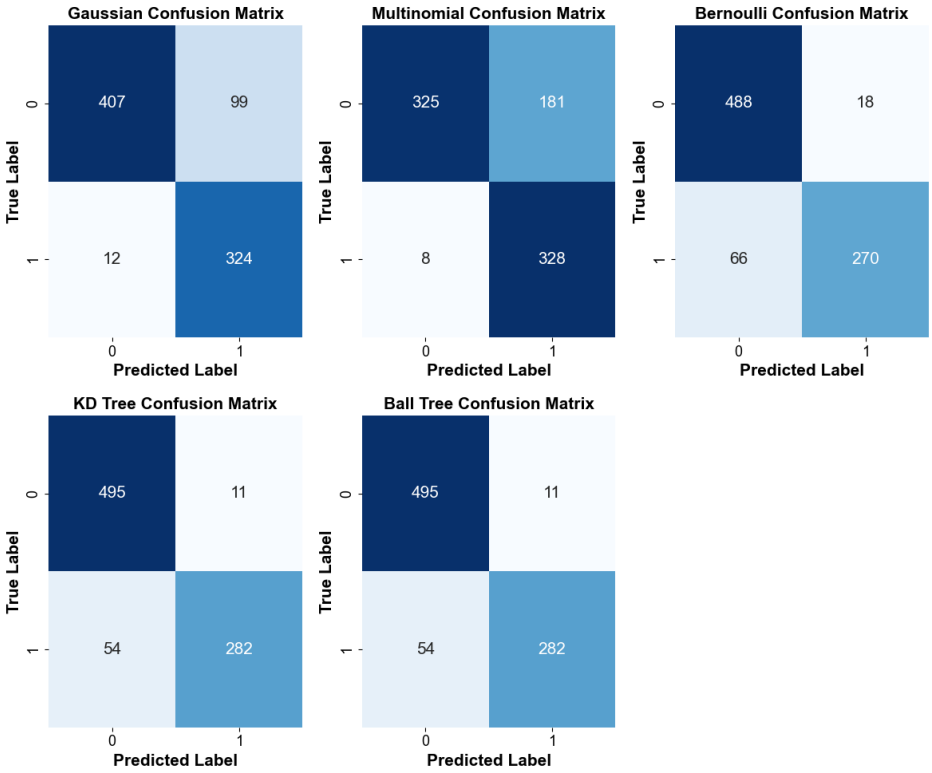

Figure 3: Confusion Matrix for Each Classifier

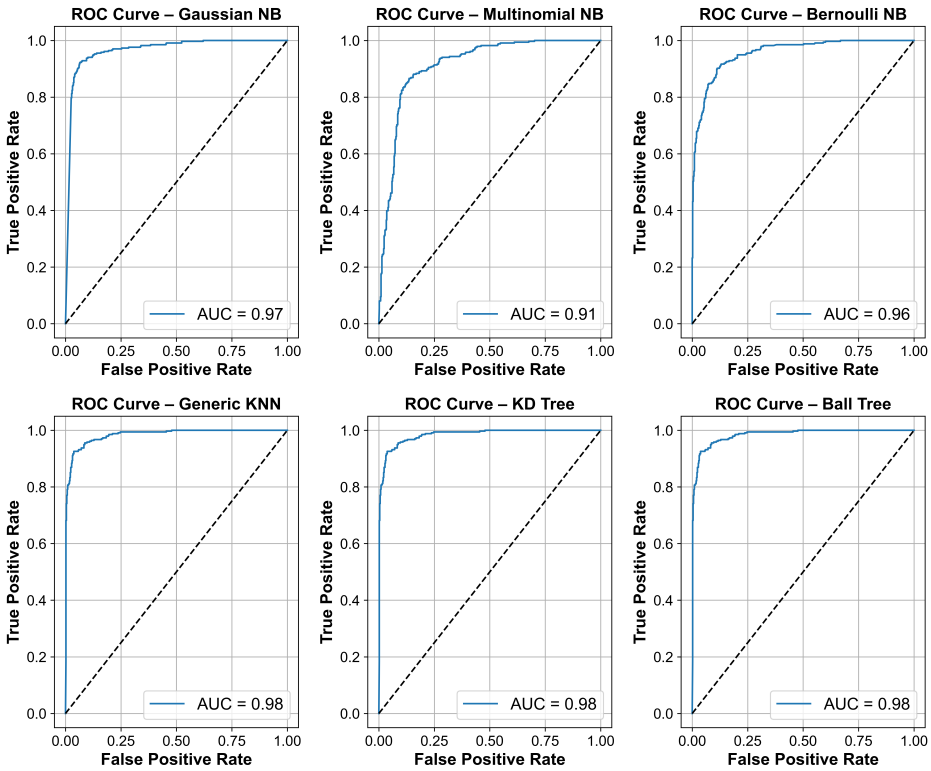

Figure 4: ROC Curves for Each Classifier

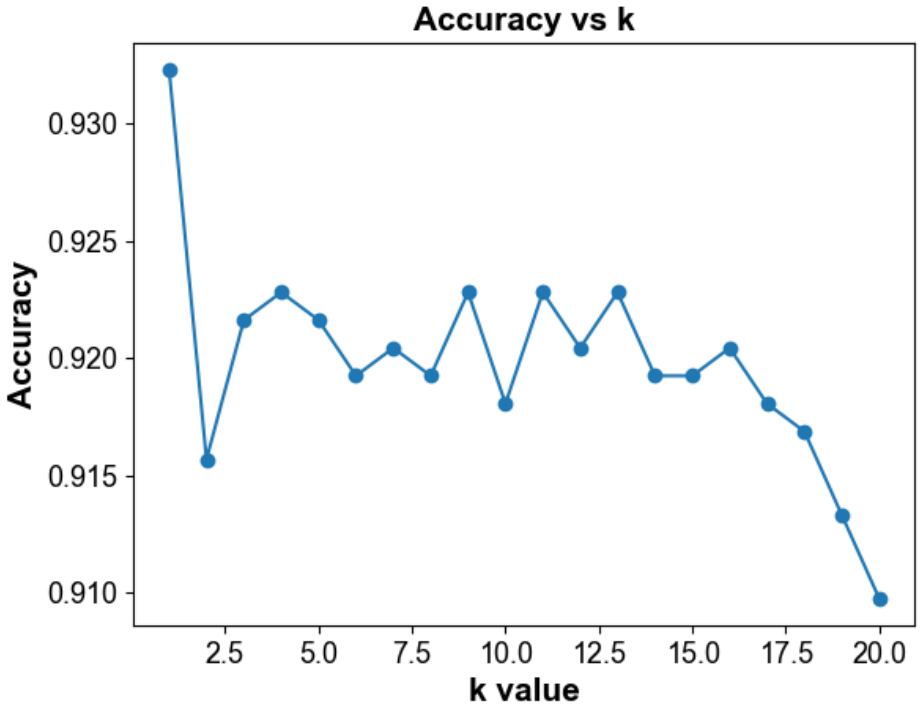

Figure 5: Accuracy vs. k Plot for KNN

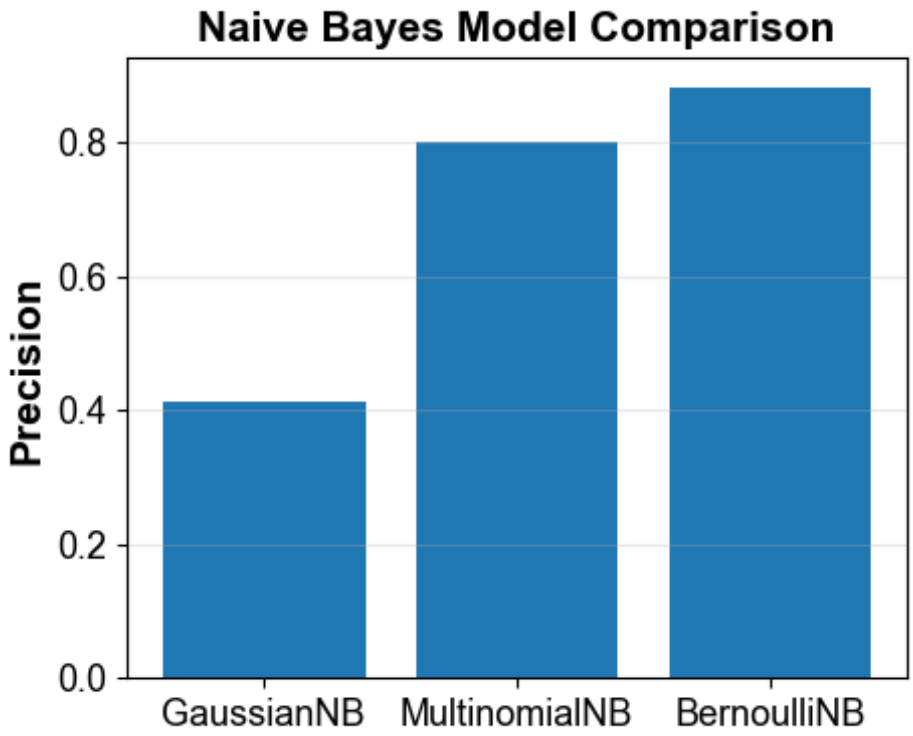

Figure 6: Performance Comparison of Naive Bayes Variants

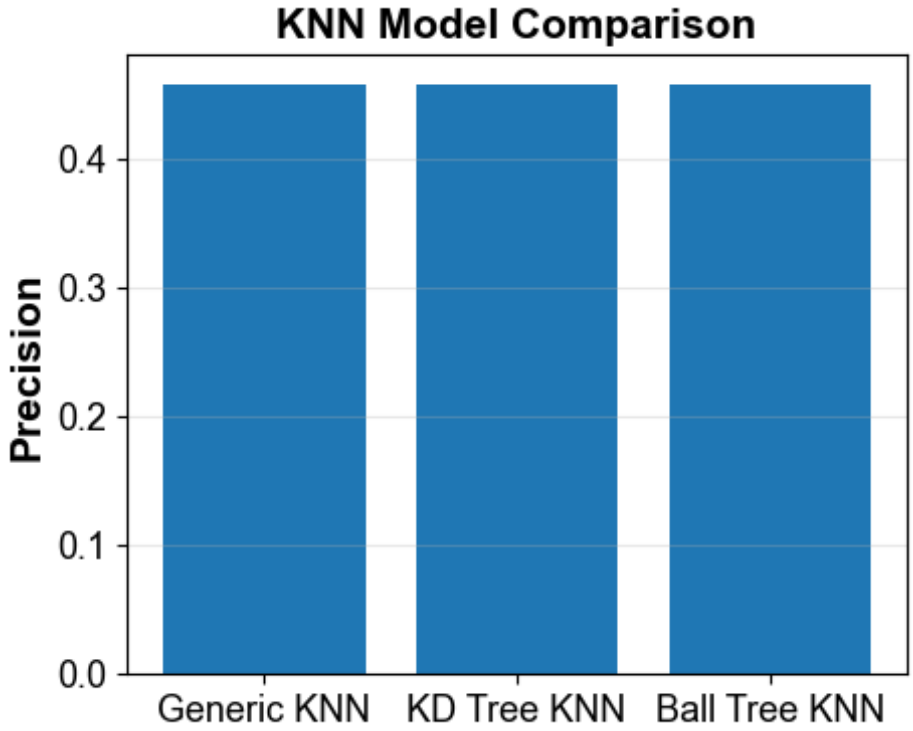

Figure 7: Performance Comparison of KNN Variants

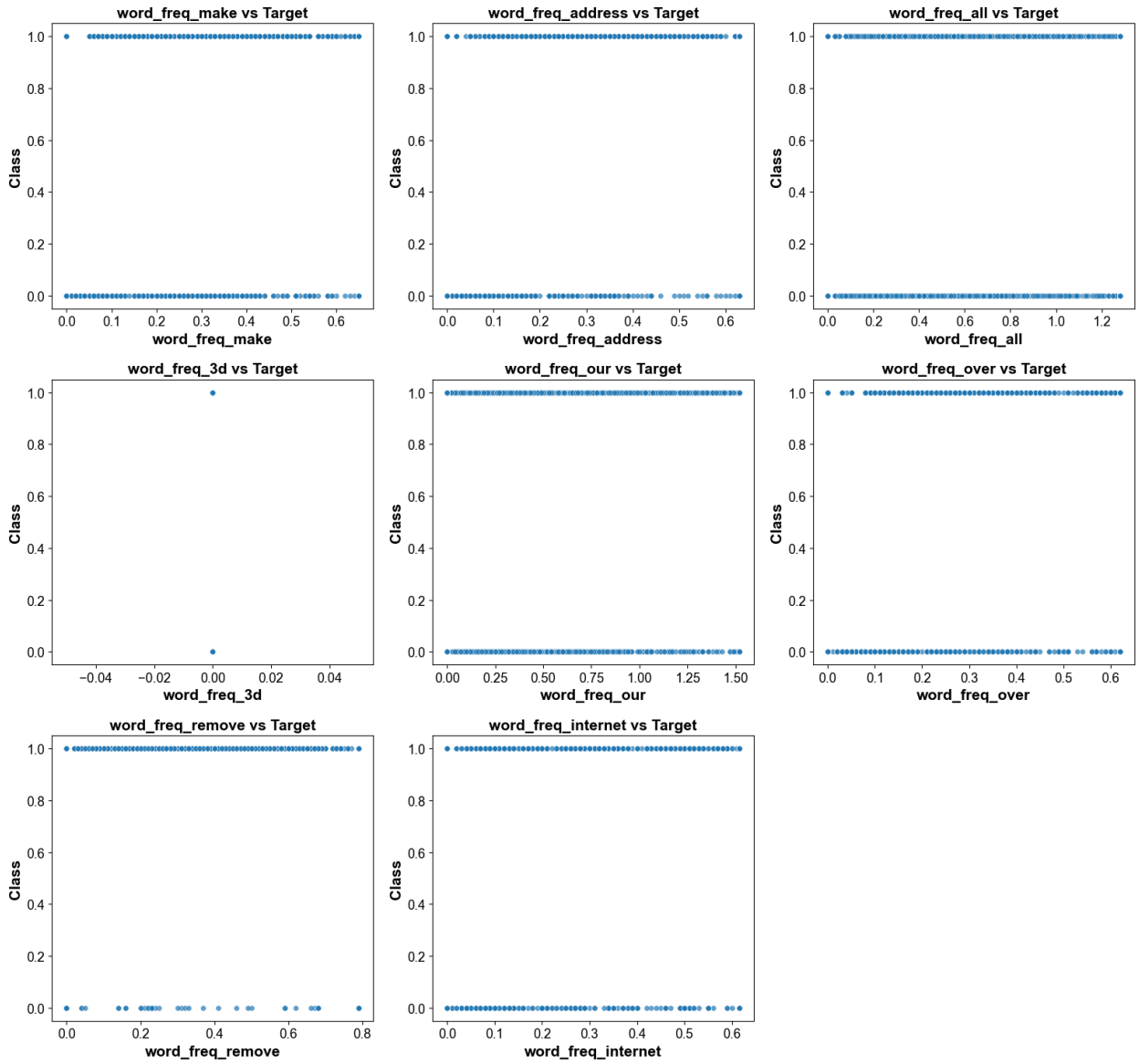

Figure 8: Feature Distribution Plots

6 Performance Tables

Naïve Bayes Performance Comparison

Table 1: Naïve Bayes Performance Metrics

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.86	0.86	0.89
Precision	0.76	0.80	0.88
Recall	0.96	0.86	0.84
F1 Score	0.85	0.83	0.86
Specificity	0.80	0.85	0.92
Training Time (s)	0.004	0.005	0.010

KNN Hyperparameter Tuning Results

Table 2: KNN Hyperparameter Tuning

Search Method	Best k	Best CV Precision	Best Parameters
Grid Search	29	0.97	metric: 'manhattan', n_neighbors: 29, weights: 'distance'
Randomized Search	29	0.97	metric: 'manhattan', n_neighbors: 29, weights: 'distance'

KNN Performance using Different Search Methods

Table 3: KNN Performance using KDTree

Metric	Value
Optimal k	29
Accuracy	0.92
Precision	0.96
Recall	0.83
F1 Score	0.89
Training Time (s)	0.02
Prediction Time (s)	0.10

Table 4: KNN Performance using BallTree

Metric	Value
Optimal k	29
Accuracy	0.92
Precision	0.96
Recall	0.83
F1 Score	0.89
Training Time (s)	0.01
Prediction Time (s)	0.07

KDTree vs BallTree Comparison

Table 5: Comparison of Neighbor Search Algorithms

Criterion	KDTree	BallTree
Accuracy	0.92	0.92
Training Time (s)	0.02	0.01
Prediction Time (s)	0.10	0.07
Memory Usage	Low / Medium	Medium / High

7 Overfitting and Underfitting Analysis

7.1 Difference between Training and Validation Accuracy

- Training accuracy measures how well the model performs on the data it was trained on, while validation accuracy evaluates performance on unseen data.
- A very high training accuracy with significantly lower validation accuracy indicates **overfitting**, where the model memorizes training data but fails to generalize.
- Similar and reasonably high training and validation accuracies indicate good **generalization**.
- Low accuracy on both training and validation datasets indicates **underfitting**, where the model is too simple to capture underlying patterns.

7.2 Effect of Small and Large Values of k

- In K-Nearest Neighbors (KNN), a **small value of** k (e.g., $k = 1$) makes the model highly sensitive to noise, leading to high variance and overfitting.
- A **large value of** k smooths the decision boundary, which may ignore important local patterns and cause underfitting.
- Choosing an optimal k balances bias and variance, resulting in better performance on unseen data.

7.3 Role of Hyperparameter Tuning in Generalization

- Hyperparameter tuning helps in selecting the best model configuration that minimizes overfitting and underfitting.
- Techniques such as Grid Search and Cross-Validation evaluate multiple parameter combinations on validation data.
- Proper tuning improves model robustness, predictive accuracy, and ensures better generalization to new datasets.

8 Bias–Variance Analysis

8.1 Bias Behavior of Naïve Bayes

- Naïve Bayes assumes strong conditional independence among features, which simplifies the learning process.
- Due to this assumption, the model has **high bias**, as it may not capture complex relationships between features.
- However, the high bias makes Naïve Bayes stable and less prone to overfitting, especially on small or noisy datasets.
- The model performs well when the independence assumption approximately holds.

8.2 Variance Behavior of KNN

- K-Nearest Neighbors (KNN) is a non-parametric and instance-based learning algorithm.
- For small values of k, KNN has **high variance** because predictions depend heavily on individual training samples.
- Noise or outliers in the training data can significantly affect model predictions.
- Increasing the value of k reduces variance by averaging over more neighbors.

8.3 Effect of Tuning on Bias–Variance Trade-off

- Hyperparameter tuning helps in finding an optimal balance between bias and variance.
- In KNN, increasing k increases bias but reduces variance, while decreasing k lowers bias but increases variance.
- Proper tuning using cross-validation improves generalization by minimizing overall prediction error.

9 Observations and Conclusion

- Naïve Bayes demonstrated fast training and prediction with low computational cost due to its simple probabilistic assumptions.
- KNN achieved competitive accuracy when an optimal value of k was selected through hyper-parameter tuning.
- Grid Search and cross-validation helped identify parameters that improved generalization and reduced overfitting.
- Naïve Bayes showed better computational efficiency, while KNN required higher memory and computation during prediction.
- Overall, both classifiers generalized well when properly tuned, with KNN offering higher flexibility and Naïve Bayes providing stable performance.